SOFTWARE REQUIREMENTS SPECIFICATION

AI-Powered Resale Marketplace

Full-stack MERN web application with an AI model layer

Frontend (React) • Backend (Node / Express) • Database (MongoDB) • AI (FastAPI)

Project	AI-Powered Resale Marketplace (working title; rename freely)
Document	Software Requirements Specification (SRS) with design and phase plan
Version / Status	1.0 / Baseline for development
Date	November 9, 2026
Prepared by	[Intern name]
Reviewed by	[Supervisor / mentor name]
Technology	MongoDB, Express.js, React, Node.js (MERN) + Python AI service

Document control

Revision history

Version	Date	Author	Description
1.0	Nov 9, 2026	[Intern name]	First complete SRS: requirements, diagrams, phase plan, AI plan

How to use this document

- **Reading order:** Sections 1-3 explain what is being built and why; Section 4-6 define requirements; Sections 7-11 define the technical design per phase; Section 12 covers the AI model; Sections 13-18 cover the schedule, testing, risks, and the project-phase handoff.
- **Starting development:** give this whole document to your AI assistant together with the phase you are starting (for example: "Start Phase 1, Frontend, using Sections 4, 9 and 13"). Section 17 lists the exact inputs and decisions to confirm first.
- **Changing scope:** change the requirement table first, then bump the version and the revision history.

Table of contents

Document control 2

1. Introduction 5

1.1 Purpose . . . 5
1.2 Scope and objectives . . . 5
1.3 Definitions, acronyms, and abbreviations . . . 6
1.4 References . . . 6
1.5 Conventions . . . 6

2. Overall description 7

2.1 Product perspective . . . 7
2.2 Product functions (summary) . . . 7
2.3 User classes and characteristics . . . 7
2.4 Operating environment . . . 7
2.5 Design and implementation constraints . . . 7
2.6 Assumptions and dependencies . . . 8
2.7 Business model (for later monetization) . . . 8

3. System architecture and diagrams 9

3.1 System architecture . . . 10
3.2 Use-case diagram . . . 11
3.3 Buyer purchase flow . . . 12
3.4 AI-assisted listing sequence . . . 13
3.5 Data model (MongoDB collections) . . . 14
3.6 AI training and inference pipeline . . . 15
3.7 Phase schedule . . . 16
3.8 Status lifecycles . . . 17

4. Functional requirements 18

4.1 Authentication and user management (7 features) . . . 18
4.2 Listing management (8 features) . . . 18
4.3 Browse and search (8 features) . . . 18
4.4 Cart and wishlist (5 features) . . . 19
4.5 Checkout and orders (7 features) . . . 19
4.6 Seller dashboard (6 features) . . . 19
4.7 Reviews and ratings (4 features) . . . 19
4.8 Notifications (4 features) . . . 19
4.9 Admin panel (7 features) . . . 20
4.10 AI module (7 features, built in Phase 4) . . . 20
4.11 Requirement summary . . . 21

5. Non-functional requirements 22

6. External interface requirements 23

6.1 User interface . . . 23
6.2 Software interfaces . . . 23
6.3 Hardware and communications . . . 23

7. Database design (MongoDB) 24

7.1 Collections and fields . . . 24
7.2 Indexes . . . 26
7.3 Data rules . . . 26

8. REST API specification 27

8.1 Conventions . . . 27
8.2 Endpoints . . . 28
8.3 Internal AI service endpoints (FastAPI, called only by the Node API) . . . 29

9. Frontend specification (Phase 1: React) 30

9.1 Technology . . . 30
9.2 Pages and routes (25) . . . 30
9.3 Reusable components . . . 31
9.4 Suggested folder structure . . . 31
9.5 Design guidelines . . . 31
9.6 Phase 1 exit criteria . . . 31

10. Backend specification (Phase 2: Node.js and Express) 32

10.1 Technology . . . 32
10.2 Architecture and folder structure . . . 32
10.3 Key backend behaviors . . . 32
10.4 Environment variables (names only) . . . 32

10.5 Phase 2 exit criteria . . . 33
10.6 Phase 3: database finalization and integration (Week 4) . . . 33

11. Deployment and DevOps **34**

11.1 Environments . . . 34
11.2 Suggested hosting (choose during Phase 5) . . . 34
11.3 CI/CD and operations . . . 34

12. AI model specification (Phase 4) **35**

12.1 Model catalog . . . 35
12.2 Datasets and data sources . . . 35
12.3 Data preparation . . . 36
12.4 Training and evaluation plan . . . 36
12.5 Serving and integration . . . 36
12.6 Monitoring and the feedback loop . . . 37
12.7 Responsible AI . . . 37

13. Development phases and plan **38**

Phase 0: Setup and planning (Week 1, days 1-2) . . . 38
Phase 1: Frontend with React (Week 1) . . . 38
Phase 2: Backend with Node.js and Express (Weeks 2-3) . . . 38
Phase 3: Database and integration (Week 4) . . . 39
Phase 4A: AI data collection and preparation (Weeks 5-6) . . . 39
Phase 4B: AI model training and evaluation (Weeks 7-9) . . . 39
Phase 4C: AI service and integration (Weeks 10-11) . . . 40
Phase 5: Testing, deployment, and documentation (Week 12) . . . 40

14. Testing and acceptance **41**

14.1 Test levels . . . 41
14.2 Key acceptance tests . . . 41
14.3 Definition of done . . . 41

15. Security checklist **42**

16. Risks and mitigation **43**

17. Development handoff guide **44**

17.1 Decisions to confirm before Phase 1 . . . 44
17.2 Prompt template to start a phase . . . 44
17.3 Rules for working phase by phase . . . 44

18. Future enhancements **45**

Appendix A: Traceability matrix (modules to phases) **46**

1. Introduction

1.1 Purpose

This document specifies the requirements, architecture, data design, AI plan, and development phases for an **AI-powered resale marketplace**: a web platform where people sell second-hand fashion items and where AI helps sellers create listings, set prices, and helps buyers find items. It is the single reference used to plan and build the system phase by phase: **frontend (React)**, **backend (Node.js / Express)**, **database (MongoDB)**, and finally the **AI model**, which is trained later on datasets.

Intended readers: the developer (AI intern), the supervisor reviewing progress, and any AI assistant that is given this document as context to help build or review the project.

1.2 Scope and objectives

The product is a consumer-to-consumer (C2C) marketplace for second-hand clothing and accessories. It starts as a complete, working web store (the first month of the internship) and then gains an AI layer that makes it different from a generic store.

- Deliver a working MERN marketplace with buyer, seller, and admin roles within the first month.
- Add AI features afterwards: photo-to-listing, price suggestion, condition grading, semantic search, similar items, and fake/duplicate detection.
- Produce a deployable, portfolio-quality product that can later be monetized (commission, promoted listings, seller subscriptions).
- Document and measure every AI model against a baseline so results are credible.

In scope (this SRS)

- Responsive React web application for buyers, sellers, and admins.
- REST API with authentication, role-based access, listings, cart, orders, reviews, notifications, and admin tools.
- MongoDB database with indexes and seed data.
- Seven AI features delivered as a separate Python (FastAPI) service called by the Node API.
- Testing, deployment, and documentation.

Out of scope (initial release)

- Real payment gateway and payouts (payment is simulated first).
- Courier / shipping integrations and live label printing.
- Real-time chat between buyer and seller.
- Native mobile apps.
- Categories other than second-hand fashion (planned for later).

1.3 Definitions, acronyms, and abbreviations

Term	Meaning
SRS	Software Requirements Specification
MERN	MongoDB, Express.js, React, Node.js stack
C2C	Consumer-to-consumer: individuals sell to other individuals
SPA	Single-page application (the React frontend)
REST / API	Representational State Transfer; HTTP interface between frontend and backend
JWT	JSON Web Token used for authentication
RBAC	Role-based access control (buyer, seller, admin)
ODM / Mongoose	Object-document mapper used to define MongoDB schemas in Node.js
CRUD	Create, Read, Update, Delete
CLIP	A pretrained model that maps images and text into one embedding space, used for semantic search and similarity
Embedding	A numeric vector that represents the meaning of an image or text
Baseline	A simple method (e.g. median price by category) that a trained model must beat
MVP	Minimum viable product: the smallest complete version that works end to end
MoSCoW	Must / Should / Could priority labels used in requirement tables

1.4 References

- ISO/IEC/IEEE 29148 (requirements engineering) and IEEE 830 (earlier SRS standard), used for the structure of this document.
- Official documentation of React, Vite, React Router, Redux Toolkit, Tailwind CSS, Node.js, Express, Mongoose, MongoDB Atlas, FastAPI, PyTorch, and Hugging Face Transformers.
- OWASP Top 10 for web application security guidance.

1.5 Conventions

- Requirement IDs use a module prefix and number (for example **LST-03**). Prefixes: AUTH, LST, BRW, CRT, ORD, SLR, REV, NTF, ADM, AI.
- Priority: **M** = Must (needed for the release), **S** = Should (important, can slip), **C** = Could (nice to have).
- "The system shall" marks a requirement. Phase tags P1 to P5 refer to the phases in Section 13.

2. Overall description

2.1 Product perspective

The system is a new, standalone web application with three tiers plus a separate AI service: a React frontend, a Node/Express REST backend backed by MongoDB, and a Python AI service. Images are stored in cloud storage. The AI service is optional at runtime: if it is unavailable, the marketplace continues to work and sellers fill the listing form manually.

2.2 Product functions (summary)

#	Module	Purpose	Features
1	Authentication and user management	Accounts, sessions, profiles, becoming a seller	7
2	Listing management	Create, edit, publish, and manage items for sale	8
3	Browse and search	Discovery: feed, categories, search, filters, detail pages	8
4	Cart and wishlist	Save and collect items before buying	5
5	Checkout and orders	Address, simulated payment, order lifecycle	7
6	Seller dashboard	Manage listings, orders, and earnings	6
7	Reviews and ratings	Trust between buyers and sellers	4
8	Notifications	In-app alerts for key events	4
9	Admin panel	Moderation and platform management	7
10	AI module	Smart listing, pricing, search, and trust features	7
	Total		**63** (56 core + 7 AI)

2.3 User classes and characteristics

User class	Description	Key needs
Buyer	Anyone browsing and purchasing second-hand items. Mostly on mobile phones.	Fast search, trustworthy listings, simple checkout, order tracking
Seller	A user who lists items. Every seller is also a buyer. Often non-technical.	Create a listing in under a minute, see orders and earnings
Admin	Platform operator (initially the developer).	Moderate content, block abusers, manage categories, review flagged items
AI Service (system actor)	Internal Python service that analyzes images and text.	Clean JSON requests, timeouts, versioned models

2.4 Operating environment

- **Client:** latest Chrome, Edge, Firefox, and Safari on desktop and mobile; camera or photo gallery for item photos.
- **Server:** Node.js LTS, MongoDB (Atlas), Python 3.10+ for the AI service, Linux containers in deployment.
- **Network:** HTTPS between all public components.

2.5 Design and implementation constraints

- The stack is MERN; the AI service is Python (FastAPI) because the ML ecosystem is Python.
- Development sequence is fixed by the company: frontend first (week 1), backend next (weeks 2-3), then database (week 4), then the AI model.
- Payment is simulated until a real gateway is added later.

- The first release covers one category (second-hand fashion) so the AI can be accurate with limited data.
- AI models are trained on datasets **after** the month-1 web application is complete. Dataset licenses must be checked before any commercial use.

2.6 Assumptions and dependencies

- The developer has access to a computer with Node.js, Git, and an internet connection; GPU access (Google Colab, Kaggle, or company hardware) is available for the AI phase.
- Third-party free tiers (MongoDB Atlas, Cloudinary, hosting) are sufficient for the demo.
- A public or self-collected image dataset of fashion items can be obtained for training.
- Each resale item is unique: quantity is always 1 and an item disappears from search when sold.

2.7 Business model (for later monetization)

Revenue stream	How it works
Commission	A small percentage of each completed sale
Promoted listings	Sellers pay to boost an item in search and the home feed
Seller subscription	Monthly plan with benefits such as unlimited AI-assisted listings and analytics
Services (later)	Authentication checks for premium items, shipping and packaging partners

3. System architecture and diagrams

This section contains the diagrams that show how the application works. Each diagram is on its own page for readability. Together they cover the system structure, users, main flows, data model, AI pipeline, schedule, and status lifecycles.

Figure	Diagram	Answers the question
3.1	System architecture	What are the parts and how do they talk to each other?
3.2	Use-case diagram	Who uses the system and what can each role do?
3.3	Buyer purchase flow	What happens from visiting the site to rating a seller?
3.4	AI-assisted listing sequence	How does a seller create a listing with AI help?
3.5	Entity-relationship (data model)	What data is stored and how is it linked?
3.6	AI training and inference pipeline	How are models built, deployed, and improved?
3.7	Phase schedule (Gantt)	When is each phase done?
3.8	Status lifecycles	What states can orders and listings move through?

3.1 System architecture

The system is a three-tier web application plus an independent AI service. The React app talks **only** to the Express API; the API enforces authentication, validation, and business rules, and is the only component that touches the database, image storage, email, and the AI service.

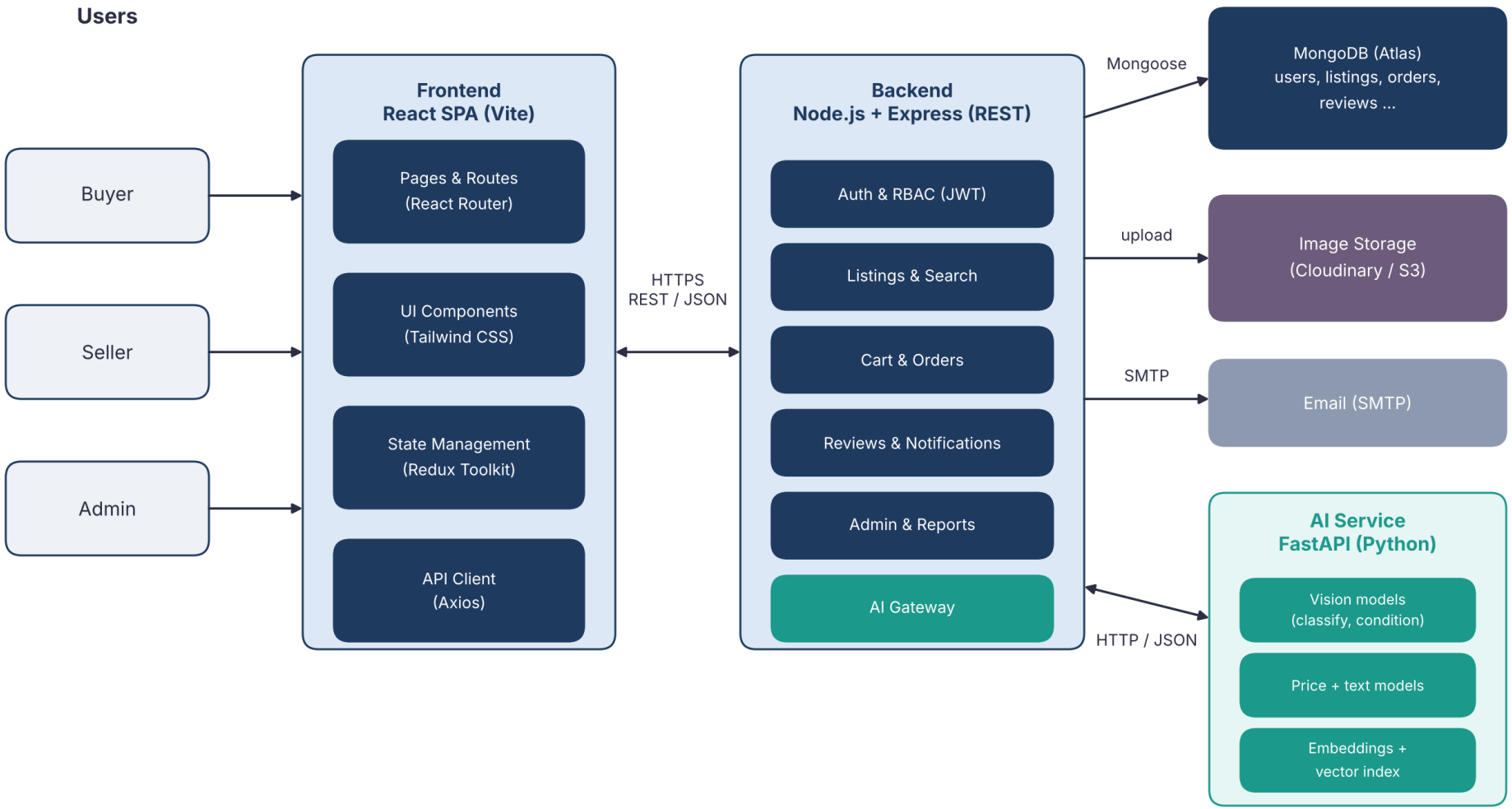


Figure 3.1: System architecture

3.2 Use-case diagram

Buyers browse, buy, and review; sellers also manage listings and orders (every seller is also a buyer); admins moderate the platform; the AI service acts as a system actor that supports listing, search, and trust features.

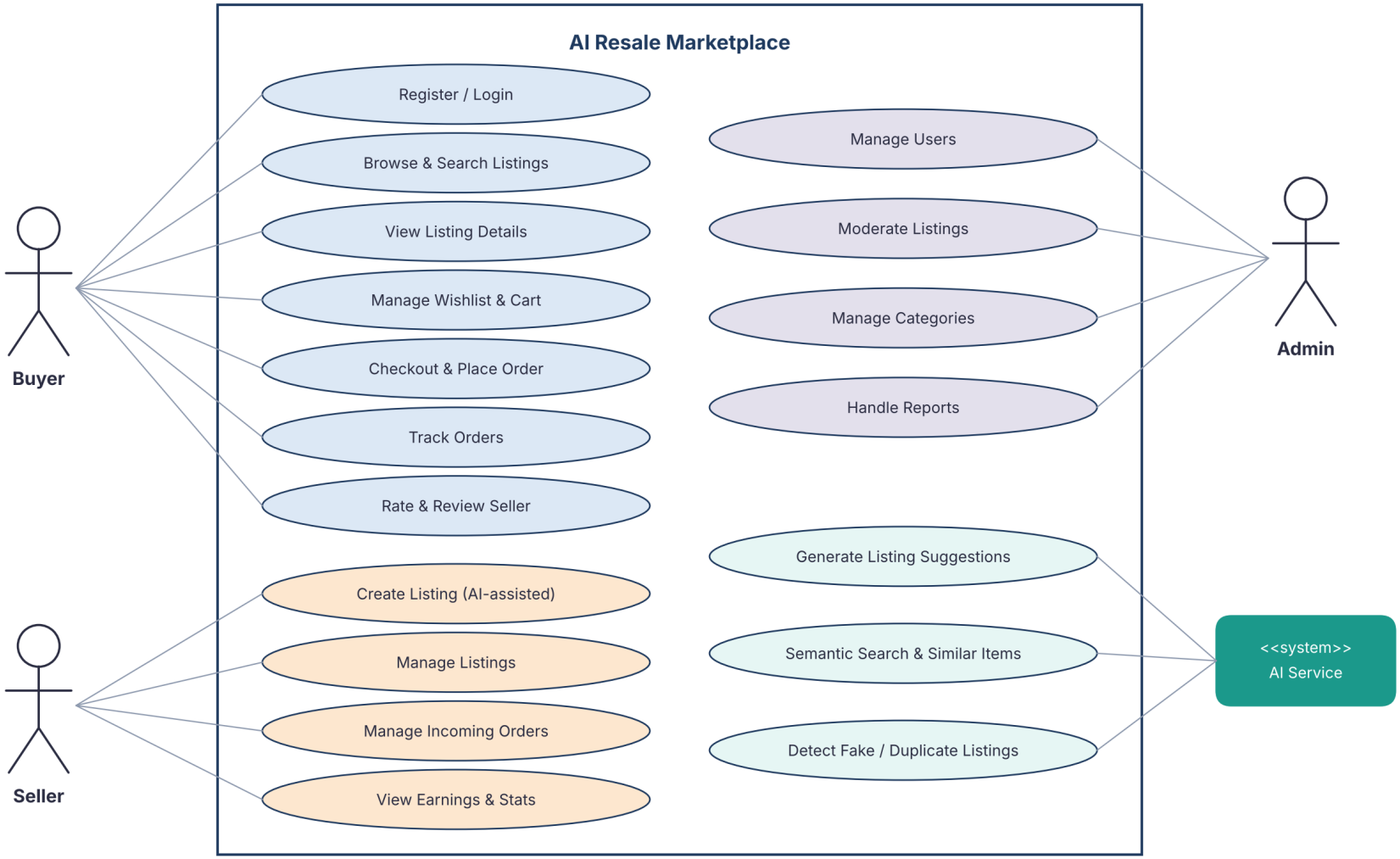


Figure 3.2: Use cases by role

3.3 Buyer purchase flow

The path from discovery to review. Guests can browse freely; login is required only at checkout. Payment is simulated in the first release.

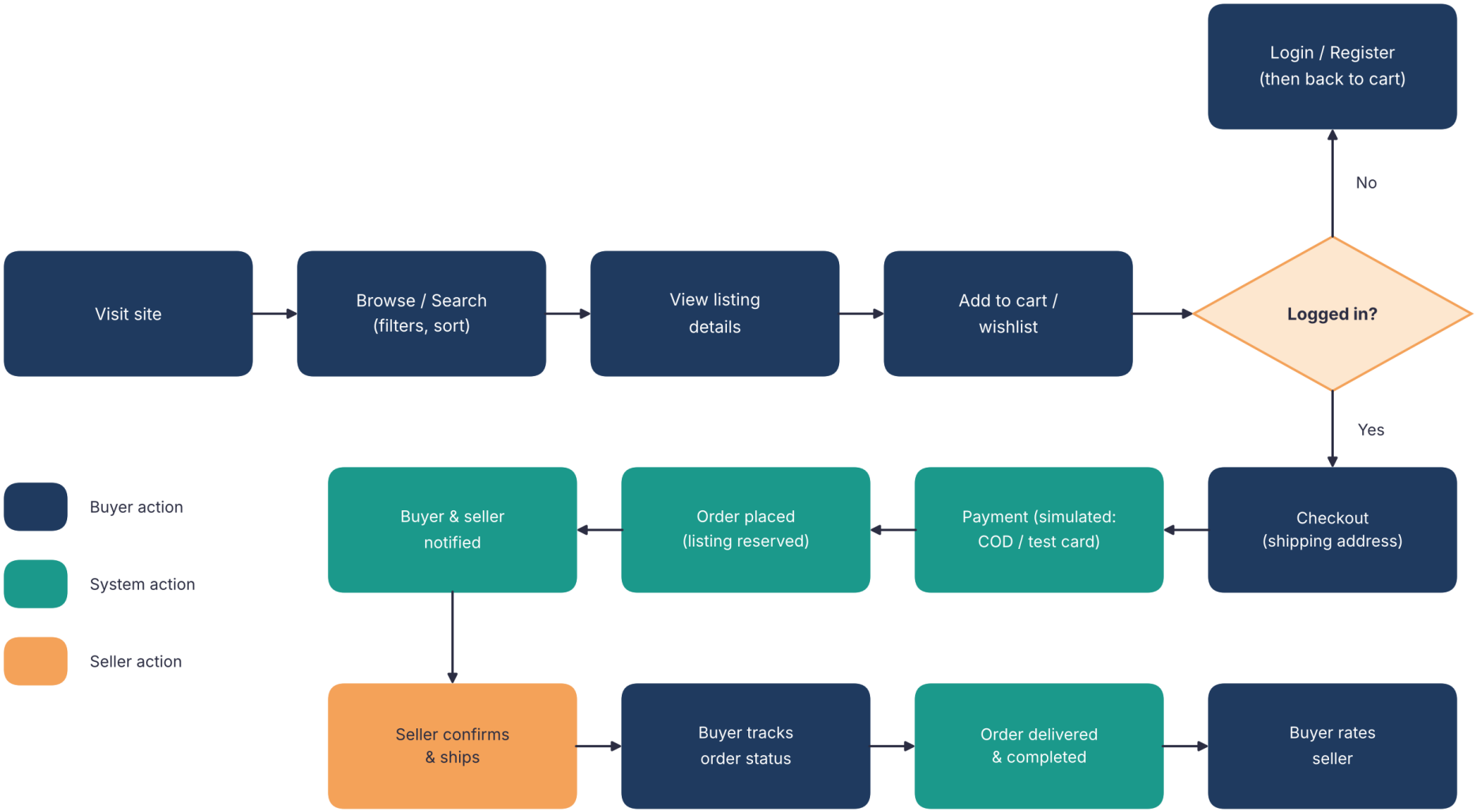


Figure 3.3: Buyer purchase flow

3.4 AI-assisted listing sequence

This is the signature feature of the product. In Phases 1-3 the endpoint in step 1 returns an empty draft (steps 4-5 and 8-9 are skipped) and the seller fills the form by hand. Because the API contract stays the same, the AI plugs in later without changing the frontend or database design.

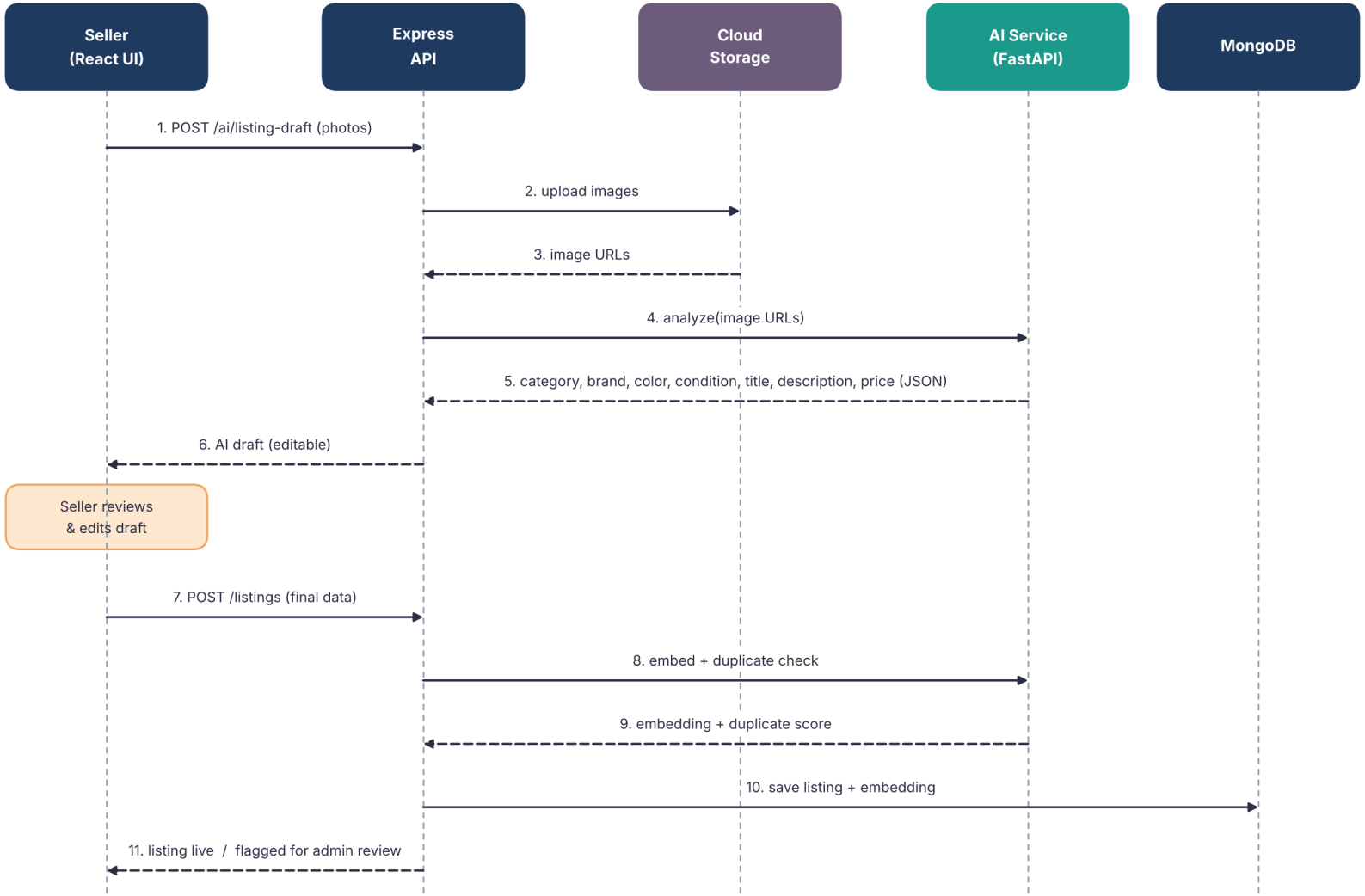


Figure 3.4: Sequence for creating a listing with AI suggestions

3.5 Data model (MongoDB collections)

Nine collections hold all data. Arrows show references (stored as ObjectIds). Order items are embedded in the order document so that the price at purchase time is preserved.

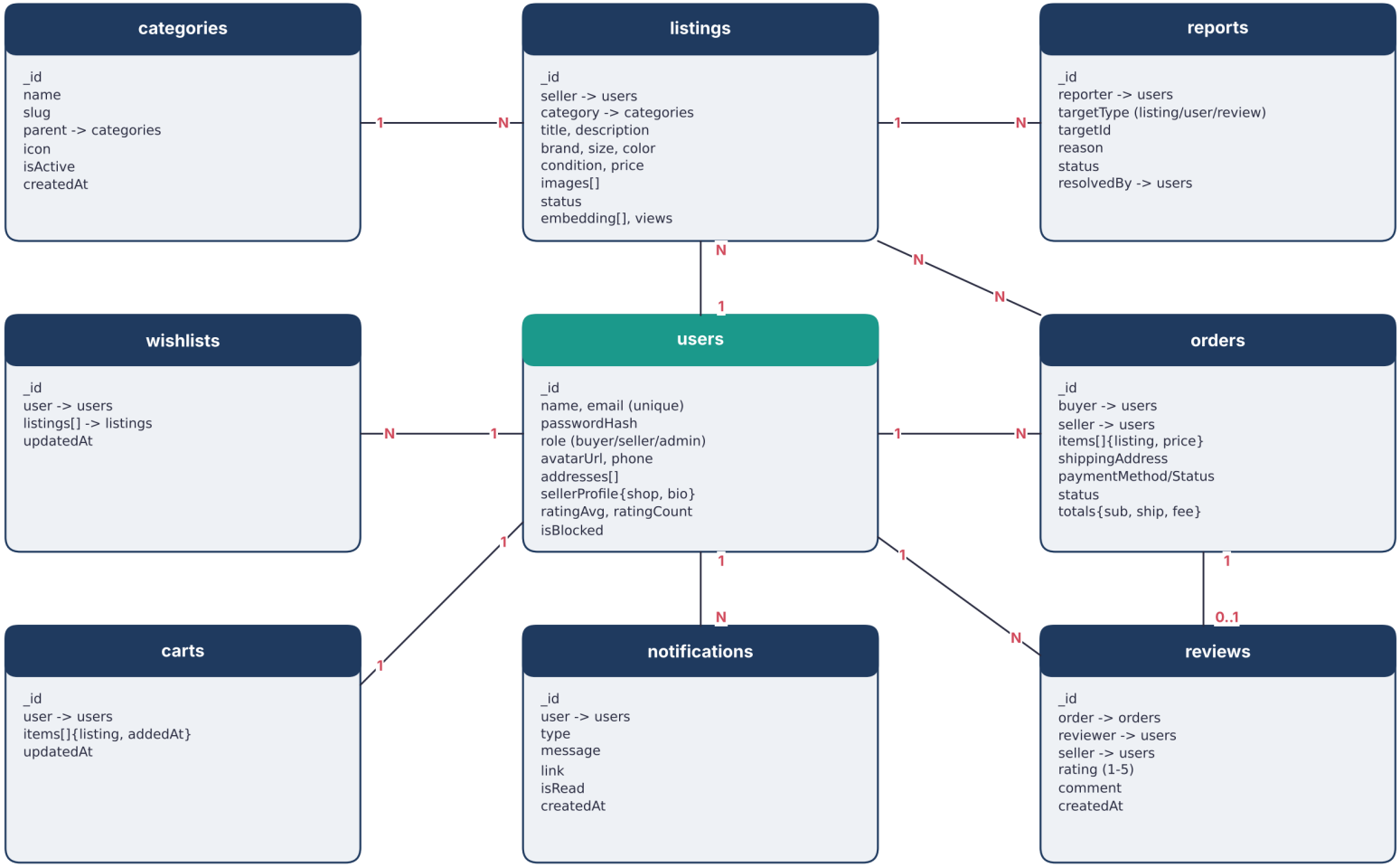


Figure 3.5: Entity-relationship view of collections (1 = one, N = many)

3.6 AI training and inference pipeline

Models are trained offline on datasets, evaluated against baselines, exported with a version, and served by the AI service. Accepted and edited suggestions from real sellers become new training data.

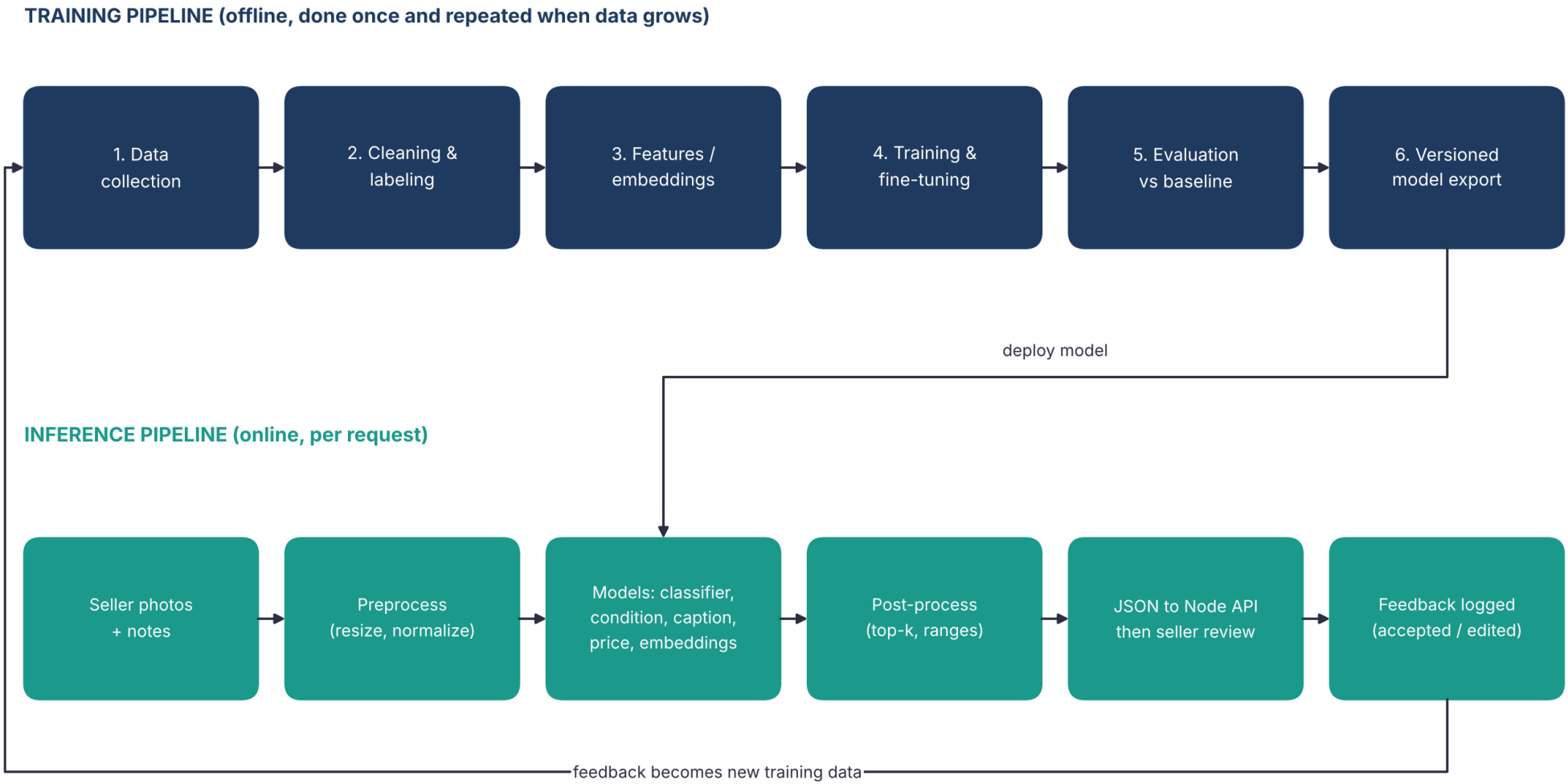


Figure 3.6: AI training and inference pipeline

3.7 Phase schedule

The first four weeks deliver the working web application. The AI phase follows and is split into data preparation, training, and integration. AI durations are proposals and can be scaled to the length of the internship.

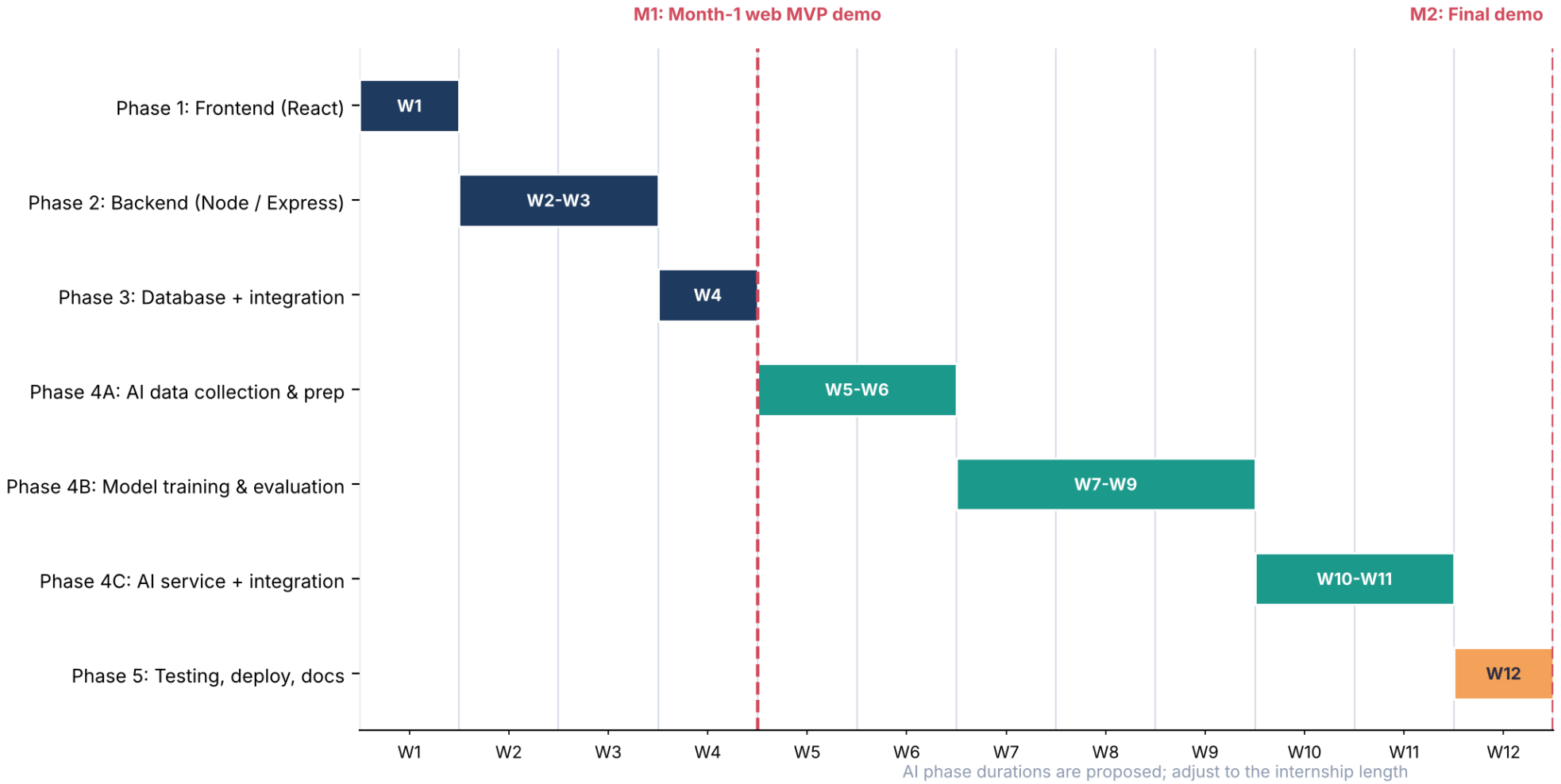


Figure 3.7: Phase schedule

3.8 Status lifecycles

Orders move forward through fixed states and can be cancelled before shipping. A listing is reserved while an order is open and returns to Active if the order is cancelled. Flagged listings are hidden from search until an admin decides.

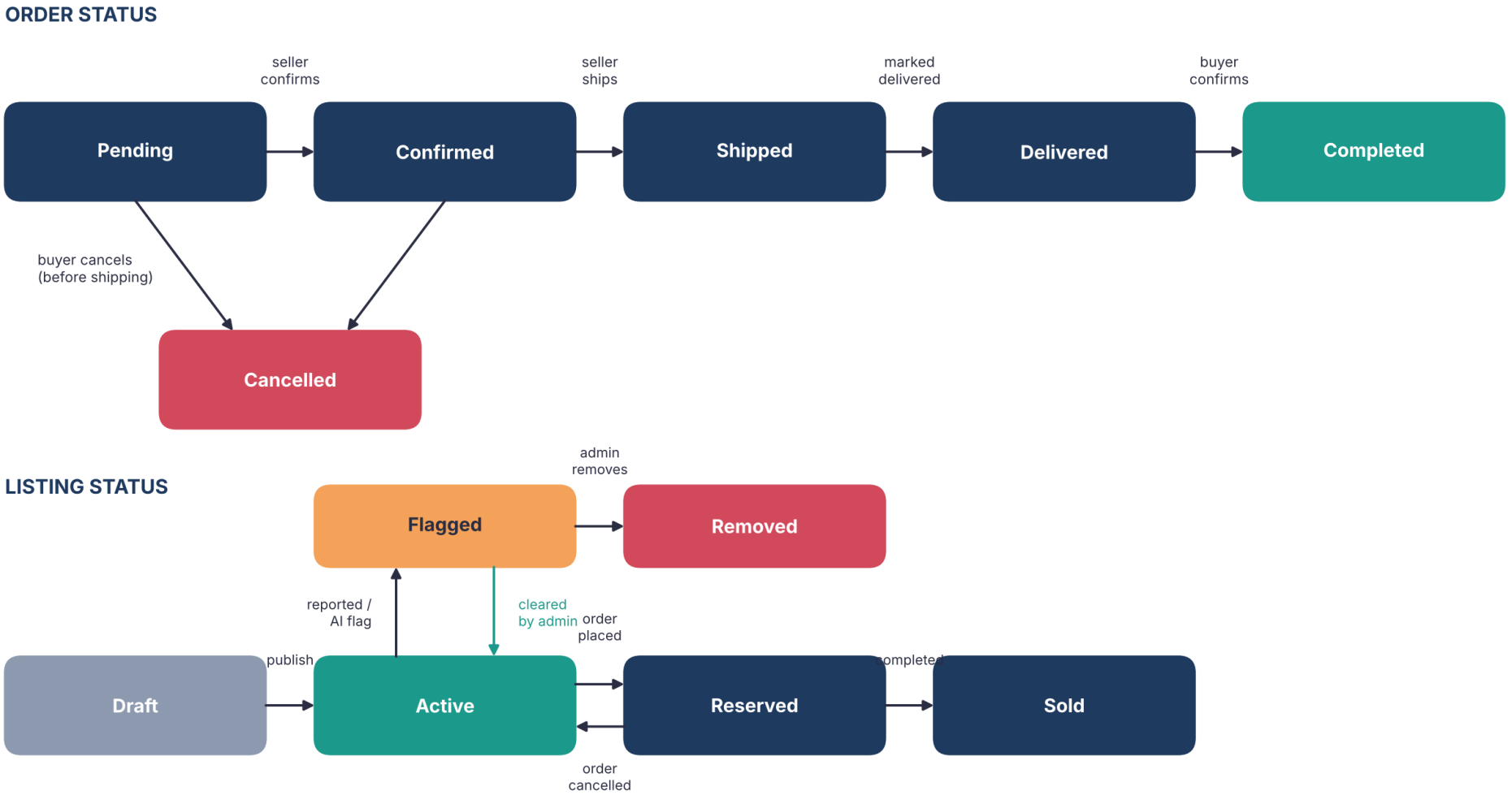


Figure 3.8: Order and listing status lifecycles

4. Functional requirements

Requirements are grouped by module. **Priority:** M = Must, S = Should, C = Could. The "system shall" wording is implied in every row. Total: 10 modules, 63 features.

4.1 Authentication and user management (7 features)

Lets people create accounts, stay signed in securely, manage their profile, and become sellers.

ID	Requirement	Pri
AUTH-01	**Register:** create an account with name, email, and password. Email must be unique; password at least 8 characters with validation feedback.	M
AUTH-02	**Login:** sign in with email and password; on success return an access token (JWT) and user profile.	M
AUTH-03	**Logout:** clear the session on the client and revoke the refresh token on the server.	M
AUTH-04	**Session handling:** short-lived access token plus refresh token; protected routes enforced on both frontend and backend; expired tokens trigger silent refresh or redirect to login.	M
AUTH-05	**Password reset:** request a reset link by email; the link token expires within 30 minutes and works once.	S
AUTH-06	**Profile management:** edit name, photo, bio, phone, and saved shipping addresses.	M
AUTH-07	**Become a seller:** a buyer enables the seller role by entering shop name and contact details.	M

4.2 Listing management (8 features)

Lets sellers publish and maintain items. In the AI phase this module gains photo-to-listing suggestions (see AI module).

ID	Requirement	Pri
LST-01	**Create listing:** title, description, category, brand, size, color, condition, and price (price greater than 0).	M
LST-02	**Image upload:** 1 to 8 images per listing (JPG, PNG, or WebP, up to 5 MB each) with preview, reorder, and cover-image selection.	M
LST-03	**Edit listing:** a seller can edit their own draft or active listing; edits are not allowed once sold.	M
LST-04	**Delete listing:** soft delete (status set to removed) so order history stays intact.	M
LST-05	**Save as draft:** store an incomplete listing and resume later.	S
LST-06	**Mark as sold:** sellers can mark an item sold elsewhere; completing an order also sets sold automatically.	M
LST-07	**Relist:** reactivate a listing that was removed, expired, or whose order was cancelled.	C
LST-08	**Validation:** required fields, length limits, allowed values, and image count are checked in the frontend and again on the server.	M

4.3 Browse and search (8 features)

How buyers discover items. Keyword search comes first; semantic search arrives in the AI phase.

ID	Requirement	Pri
BRW-01	**Home feed:** new arrivals, featured categories, and popular items.	M
BRW-02	**Category browsing:** listing grid per category and subcategory.	M
BRW-03	**Keyword search:** match title, brand, and description; results ranked by relevance.	M
BRW-04	**Filters:** price range, size, condition, brand, color, and category; filters combine and are reflected in the URL.	M
BRW-05	**Sorting:** newest, price low to high, price high to low, most popular.	M
BRW-06	**Pagination or infinite scroll:** 20 items per page with loading and empty states.	M
BRW-07	**Listing detail page:** image gallery, details, seller card, related items, add to cart, add to wishlist.	M
BRW-08	**Seller public profile:** seller info, rating, and their active listings.	S

4.4 Cart and wishlist (5 features)

Because each item is unique, cart items have quantity 1 and cannot be added if sold or owned by the buyer.

ID	Requirement	Pri
CRT-01	**Add to cart:** logged-in buyers add an active listing; own or sold items are rejected with a clear message.	M
CRT-02	**Remove from cart:** remove an item at any time.	M
CRT-03	**Cart summary:** items, subtotal, flat shipping fee, total; items that became unavailable are flagged.	M
CRT-04	**Favorites:** add or remove a listing from the wishlist from cards and detail pages.	S
CRT-05	**View wishlist:** a page listing saved items with a move-to-cart action.	S

4.5 Checkout and orders (7 features)

Order placement must be safe against two buyers purchasing the same unique item at the same time.

ID	Requirement	Pri
ORD-01	**Checkout address:** choose a saved address or enter a new one; validate required fields.	M
ORD-02	**Simulated payment:** cash on delivery and a test-mode card option that return success or failure.	M
ORD-03	**Place order:** create one order per seller; mark listings reserved in one atomic operation so an item cannot be sold twice.	M
ORD-04	**Order confirmation:** confirmation page and notification to buyer and seller.	M
ORD-05	**Order history:** buyer sees all orders with status and can filter by status.	M
ORD-06	**Order tracking:** status timeline Pending, Confirmed, Shipped, Delivered, Completed.	M
ORD-07	**Cancel order:** buyer may cancel before shipping; the listing returns to Active.	S

4.6 Seller dashboard (6 features)

Gives sellers control over inventory and sales.

ID	Requirement	Pri
SLR-01	**My listings:** list with status, views, and favorites count; quick actions to edit or mark sold.	M
SLR-02	**Incoming orders:** list and detail view of orders for the seller's items.	M
SLR-03	**Update order status:** confirm, ship (with an optional tracking note), and mark delivered.	M
SLR-04	**Earnings summary:** total sales, platform commission, and net earnings.	S
SLR-05	**Basic sales stats:** sales over time and most-viewed listings.	C
SLR-06	**Seller settings:** shop name, description, contact details, and return policy text.	S

4.7 Reviews and ratings (4 features)

Ratings build trust between strangers, which is essential for a resale marketplace.

ID	Requirement	Pri
REV-01	**Rate seller:** 1 to 5 stars after an order is completed; one rating per order.	M
REV-02	**Write review:** optional comment up to 500 characters.	S
REV-03	**View reviews:** paginated reviews on the seller profile.	S
REV-04	**Average rating:** computed automatically and shown with the review count.	M

4.8 Notifications (4 features)

Simple in-app notifications; email notifications are a later enhancement.

ID	Requirement	Pri
NTF-01	**Notification center:** list of notifications with an unread badge in the navbar.	S
NTF-02	**Order placed alert:** to both buyer and seller.	S
NTF-03	**Order status alert:** sent to the buyer whenever the status changes.	S
NTF-04	**Mark as read:** mark one or all notifications as read.	S

4.9 Admin panel (7 features)

Basic moderation tools to keep the marketplace safe.

ID	Requirement	Pri
ADM-01	**Dashboard overview:** totals for users, listings, orders, revenue, and flagged items.	S
ADM-02	**Manage users:** search and view users.	M
ADM-03	**Block / unblock user:** blocked users cannot log in or list items.	M
ADM-04	**Manage listings:** search and filter all listings including flagged ones.	M
ADM-05	**Remove listing:** remove with a reason; the seller is notified.	M
ADM-06	**Manage categories:** create, rename, reorder, and deactivate categories.	M
ADM-07	**Handle reports:** review user reports on listings, users, or reviews; dismiss or act.	S

4.10 AI module (7 features, built in Phase 4)

Each AI feature is assistive: the user can always edit or ignore it, and the app works if the AI service is down. Details, models, and metrics are in Section 12.

ID	Requirement	Pri
AI-01	**Semantic search:** natural-language queries such as "vintage-looking leather jacket under 5000" using text and image embeddings.	M
AI-02	**Similar items:** "More like this" on the detail page using image-embedding similarity.	M
AI-03	**Photo to attributes:** predict category, color, and brand from photos; show top suggestions with confidence.	M
AI-04	**Auto title and description:** generate an editable draft from the photo attributes and seller notes.	M
AI-05	**Condition grading:** predict condition class (new with tags, like new, good, fair) with confidence.	S
AI-06	**Price suggestion:** suggest a price and a reasonable range based on similar items.	M
AI-07	**Fake / duplicate detection:** flag near-duplicate or reused photos and extreme price outliers for admin review.	S

4.11 Requirement summary

Module	Must	Should	Could	Total
Authentication and users	6	1	0	7
Listing management	6	1	1	8
Browse and search	7	1	0	8
Cart and wishlist	3	2	0	5
Checkout and orders	6	1	0	7
Seller dashboard	3	2	1	6
Reviews and ratings	2	2	0	4
Notifications	0	4	0	4
Admin panel	5	2	0	7
AI module	5	2	0	7
Total	**43**	**18**	**2**	**63**

5. Non-functional requirements

ID	Category	Requirement	How to verify
NFR-01	Performance	Main pages load in under 3 seconds on a normal broadband or 4G connection; images are lazy-loaded and resized.	Lighthouse / browser dev-tools
NFR-02	Performance	Standard API responses in under 500 ms for 95% of requests under demo load.	API timing tests
NFR-03	Performance	AI listing analysis completes in under 5 seconds; semantic search in under 1.5 seconds.	AI service timing logs
NFR-04	Security	Passwords hashed with bcrypt; JWT with expiry; refresh tokens revocable; HTTPS only in production.	Code review, auth tests
NFR-05	Security	Role-based access control on every protected endpoint; users can modify only their own data.	Negative API tests per role
NFR-06	Security	All input validated and sanitized (injection and XSS protection); rate limiting on login and write endpoints; secure HTTP headers.	Security checklist (Section 15)
NFR-07	Security	Uploads restricted by type and size; filenames not trusted; images scanned for valid format.	Upload tests
NFR-08	Reliability	Friendly error messages; the app still works when the AI service is unavailable (manual listing form); regular database backups.	Failure-mode tests
NFR-09	Data integrity	An item cannot be sold twice; order creation is atomic; validation exists on both frontend and backend.	Concurrency test
NFR-10	Scalability	Stateless API (horizontally scalable); database indexes for search and filters; pagination everywhere.	Index review, load test
NFR-11	Usability	Mobile-first responsive design; consistent navigation; loading, empty, and error states on every data view.	Manual UI review
NFR-12	Accessibility	Alt text on images, sufficient color contrast, keyboard-usable controls, labels on form fields.	Lighthouse accessibility
NFR-13	Compatibility	Works in the latest versions of Chrome, Edge, Firefox, and Safari (desktop and mobile).	Cross-browser check
NFR-14	Maintainability	Modular folder structure, consistent REST naming, environment variables for configuration, Git with meaningful commits, README and API documentation.	Code review
NFR-15	Testability	Core backend logic covered by automated tests; critical user flows covered by end-to-end tests.	Test report
NFR-16	AI quality	Each model has a written metric, a baseline, and a test-set result; suggestions are always editable and labeled "AI suggested".	Model cards (Section 12)
NFR-17	Privacy	Collect only needed personal data; strip location metadata from uploaded photos; users can edit or delete their account data.	Privacy checklist
NFR-18	Availability	Demo environment available during working hours; health-check endpoints for backend and AI service.	Uptime check
NFR-19	Observability	Request logging with correlation IDs; error logging; AI request logging (without storing personal data).	Log review

6. External interface requirements

6.1 User interface

- Mobile-first responsive layout with breakpoints at about 640, 768, 1024, and 1280 pixels.
- Consistent design system: color tokens, typography scale, spacing scale, reusable components.
- The **Create listing** screen is the signature experience: upload photos first, then the form is prefilled (by AI in Phase 4) and editable.
- Visible loading skeletons, empty states, error banners, and toast messages for success and failure.

6.2 Software interfaces

Interface	Purpose	Protocol
Frontend to Backend API	All application data	HTTPS, JSON, REST
Backend to MongoDB	Persistent storage via Mongoose	MongoDB driver (TLS)
Backend to Image storage	Store and serve item images (Cloudinary or S3-compatible)	HTTPS API
Backend to Email service	Password reset and later notifications	SMTP or provider API
Backend to AI service	Image analysis, embeddings, price suggestion	HTTP, JSON, internal token
Payment gateway (later)	Real payments	Provider API (out of scope for v1)

6.3 Hardware and communications

- No special hardware for end users; a phone camera or gallery is used for photos.
- AI training uses a GPU (Colab, Kaggle, or company hardware); AI inference can run on CPU for the demo.
- All public traffic uses HTTPS; the AI service is reachable only from the backend (private network or shared secret).

7. Database design (MongoDB)

MongoDB Atlas with Mongoose schemas. References use ObjectId; data that must not change after the fact (such as the price at purchase) is embedded. Every collection has **createdAt** and **updatedAt** timestamps. The schema is defined here so backend work in Phase 2 can start without rework; Phase 3 finalizes indexes, seed data, and the connection of the frontend to real data.

7.1 Collections and fields

users

Field	Type	Notes
name	String	Required
email	String	Unique, lowercase, indexed
passwordHash	String	bcrypt hash; never returned by the API
role	String	buyer | seller | admin (default buyer)
avatarUrl, phone	String	Optional
addresses	[Object]	label, line1, city, region, postalCode, country, isDefault
sellerProfile	Object	shopName, bio, returnPolicy (present when role is seller)
ratingAvg, ratingCount	Number	Updated when a review is added
isBlocked	Boolean	Blocks login and listing
refreshTokens	[String]	Hashed refresh tokens for revocation

categories

Field	Type	Notes
name, slug	String	slug unique
parent	ObjectId	Optional reference to categories for subcategories
icon	String	Optional
isActive	Boolean	Inactive categories are hidden

listings

Field	Type	Notes
seller	ObjectId	Reference to users; indexed
category	ObjectId	Reference to categories; indexed
title, description	String	Text index for keyword search
brand, size, color	String	Filterable
condition	String	new_with_tags | like_new | good | fair
price	Number	Greater than 0
images	[Object]	url, publicId, isCover, order
status	String	draft | active | reserved | sold | flagged | removed
embedding	[Number]	Image/text vector (Phase 4); optional until then
aiMeta	Object	Suggested values, model version, accepted or edited flags (Phase 4)
views, favoritesCount	Number	Counters for popularity sorting

orders

Field	Type	Notes
buyer, seller	ObjectId	References to users; indexed
items	[Object]	listing (ObjectId), title, image, priceAtPurchase
shippingAddress	Object	Copied at checkout so later profile edits do not change it
paymentMethod, paymentStatus	String	cod | test_card; pending | paid | failed
status	String	pending | confirmed | shipped | delivered | completed | cancelled
statusHistory	[Object]	status, at, note (powers the tracking timeline)
totals	Object	subtotal, shipping, platformFee, total

carts, wishlists

Field	Type	Notes
user	ObjectId	Unique per user
items / listings	[Object] / [ObjectId]	Cart items hold listing and addedAt; wishlist holds listing ids

reviews

Field	Type	Notes
order	ObjectId	Unique: one review per order
reviewer, seller	ObjectId	References to users
rating	Number	1 to 5
comment	String	Up to 500 characters

notifications

Field	Type	Notes
user	ObjectId	Recipient; indexed
type	String	order_placed | order_status | listing_removed | system
message, link	String	Text and in-app route
isRead	Boolean	Default false

reports

Field	Type	Notes
reporter	ObjectId	Reference to users
targetType, targetId	String / ObjectId	listing | user | review
reason	String	Predefined reasons plus free text
status	String	open | dismissed | actioned
resolvedBy	ObjectId	Admin

7.2 Indexes

Collection	Index	Why
users	email (unique)	Login and duplicate prevention
listings	text(title, brand, description)	Keyword search
listings	(status, category, price), (status, createdAt desc), (seller, status)	Browse, filters, sorting, seller dashboard
orders	(buyer, createdAt desc), (seller, status, createdAt desc)	Order history and seller order list
reviews	order (unique), (seller, createdAt desc)	One review per order; seller profile
notifications	(user, isRead, createdAt desc)	Notification center
carts, wishlists	user (unique)	One document per user

7.3 Data rules

- An item may be in many carts but can be reserved by only one order; reservation uses an atomic conditional update (status active to reserved) inside a transaction.
- Deleting a user or listing is a soft delete so past orders and reviews stay valid.
- Seed data: at least 3 categories with subcategories, 20 users, 150 listings with real-looking photos, and sample orders and reviews for demos and for the first AI experiments.
- Backups: automated Atlas backups or a scheduled export; the seed script must recreate a clean demo database in one command.

8. REST API specification

Base path **/api**. JSON request and response bodies. Authentication via **Authorization: Bearer <access token>**. Access column: **Public**, **User** (any logged-in user), **Seller**, **Admin**.

8.1 Conventions

- **Success:** {"success": true, "data": ...}. **Error:** {"success": false, "error": {"code": "...", "message": "..."}} with a matching HTTP status.
- **Status codes:** 200 OK, 201 Created, 400 validation error, 401 not authenticated, 403 not allowed, 404 not found, 409 conflict (for example item already sold), 429 too many requests, 500 server error.
- **Lists** accept page, limit, sort and return {items, page, limit, total}. Search/filter parameters: q, category, minPrice, maxPrice, size, condition, brand, color, sort.
- **Validation** runs in middleware before controllers; the same rules are mirrored in the frontend forms.

8.2 Endpoints

Method	Endpoint	Access	Description
POST	/auth/register	Public	Create account
POST	/auth/login	Public	Login, returns tokens
POST	/auth/refresh	Public	Exchange refresh token for new access token
POST	/auth/logout	User	Revoke refresh token
POST	/auth/forgot-password	Public	Send reset link
POST	/auth/reset-password	Public	Set new password with token
GET	/users/me	User	Current profile
PATCH	/users/me	User	Update profile and addresses
POST	/users/me/become-seller	User	Enable seller role
GET	/users/:id	Public	Public seller profile
GET	/categories	Public	Category tree
GET	/listings	Public	Search, filter, sort, paginate
GET	/listings/:id	Public	Listing detail (increments views)
GET	/listings/:id/similar	Public	Similar items (AI phase; category-based before that)
POST	/listings	Seller	Create listing (draft or active)
PATCH	/listings/:id	Seller	Edit own listing
DELETE	/listings/:id	Seller	Soft delete own listing
POST	/listings/:id/images	Seller	Upload images (multipart)
POST	/listings/:id/mark-sold	Seller	Mark sold elsewhere
GET	/seller/listings	Seller	My listings with stats
GET	/cart	User	Get cart
POST	/cart/items	User	Add listing to cart
DELETE	/cart/items/:listingId	User	Remove from cart
GET	/wishlist	User	Get wishlist
POST	/wishlist/:listingId	User	Add favorite
DELETE	/wishlist/:listingId	User	Remove favorite
POST	/orders	User	Checkout: create orders from cart (atomic)
GET	/orders	User	Buyer order history
GET	/orders/:id	User	Order detail with status history
POST	/orders/:id/cancel	User	Cancel before shipping
GET	/seller/orders	Seller	Incoming orders
PATCH	/seller/orders/:id/status	Seller	Confirm, ship, deliver
GET	/seller/summary	Seller	Earnings and stats
POST	/orders/:id/review	User	Rate seller after completion
GET	/users/:id/reviews	Public	Seller reviews
GET	/notifications	User	List notifications
PATCH	/notifications/:id/read	User	Mark one read
PATCH	/notifications/read-all	User	Mark all read
POST	/reports	User	Report a listing, user, or review
GET	/admin/stats	Admin	Dashboard totals
GET	/admin/users	Admin	Search users

Method	Endpoint	Access	Description
PATCH	/admin/users/:id/block	Admin	Block or unblock
GET	/admin/listings	Admin	All listings incl. flagged
DELETE	/admin/listings/:id	Admin	Remove with reason
POST / PATCH / DELETE	/admin/categories[/:id]	Admin	Manage categories
GET	/admin/reports	Admin	Open reports
PATCH	/admin/reports/:id	Admin	Dismiss or action
POST	/ai/listing-draft	Seller	Photos to draft listing (empty draft before Phase 4)
GET	/ai/search?q=	Public	Semantic search (falls back to keyword)
POST	/ai/price-suggestion	Seller	Suggested price and range

8.3 Internal AI service endpoints (FastAPI, called only by the Node API)

Method	Endpoint	Description
GET	/health	Liveness and model versions
POST	/analyze	Image URLs in; category, color, brand, condition, caption, and title/description draft out
POST	/price	Listing attributes in; suggested price and range out
POST	/embed	Image URL or text in; embedding vector out
POST	/search	Query text in; ranked listing ids out (vector similarity)
POST	/check-duplicate	Image URL in; duplicate score and nearest matches out

9. Frontend specification (Phase 1: React)

9.1 Technology

Concern	Choice	Notes
Build tool	React 18 + Vite	Fast dev server and builds
Routing	React Router	Nested and protected routes
State	Redux Toolkit (auth, cart, UI)	Server data may use RTK Query or TanStack Query
Styling	Tailwind CSS	Design tokens in config; mobile first
Forms	React Hook Form + Zod (or Yup)	Same rules as backend validation
HTTP	Axios with interceptors	Attaches token, refreshes on 401
Icons / charts	Lucide icons, Recharts	Dashboard charts
Mock data (week 1)	JSON files or Mock Service Worker	Replaced by real API in Phase 3

9.2 Pages and routes (25)

Route	Page	Access	Week-1 priority
/	Home	Public	Must
/search	Search results	Public	Must
/category/:slug	Category page	Public	Must
/listing/:id	Listing detail	Public	Must
/seller/:id	Seller public profile	Public	Should
/login, /register	Login, Register	Public	Must
/forgot-password	Forgot / reset password	Public	Could
/cart	Cart	User	Must
/checkout	Checkout	User	Must
/order/confirmation/:id	Order confirmation	User	Must
/orders	My orders	User	Should
/orders/:id	Order detail and tracking	User	Should
/wishlist	Wishlist	User	Should
/account	Profile settings	User	Should
/notifications	Notifications	User	Could
/sell/new	Create listing	Seller	Must
/sell/:id/edit	Edit listing	Seller	Should
/seller/dashboard	Seller dashboard	Seller	Must
/seller/orders	Seller orders	Seller	Should
/admin	Admin dashboard	Admin	Later
/admin/users	Admin users	Admin	Later
/admin/listings	Admin listings	Admin	Later
/admin/categories	Admin categories and reports	Admin	Later
*	Not found (404)	Public	Must

Realistic scope for week 1: build the Must and Should pages for buyers and sellers (about 16) with mock data and a reusable component library. Admin pages depend heavily on real data and are built after the backend exists.

9.3 Reusable components

- **Layout:** Navbar (search, cart badge, notification badge, user menu), Footer, PageContainer, ProtectedRoute and RoleRoute.
- **Listings:** ListingCard, ListingGrid, FilterSidebar / FilterDrawer, SortSelect, SearchBar, ImageGallery, Pagination.
- **Selling:** ImageUploader (drag and drop, reorder, preview), ListingForm, AIDraftPanel (placeholder in Phase 1, wired to AI in Phase 4), ConditionSelect, PriceInput.
- **Commerce:** CartItem, OrderSummary, AddressForm, PaymentOptions, OrderTimeline, StatusBadge.
- **Shared UI:** Button, Input, Select, Modal, Toast, Skeleton, EmptyState, ErrorBanner, RatingStars, Tabs, Badge.

9.4 Suggested folder structure

```
frontend/
  src/
    app/            store.js, routes.jsx
    api/            axios client, one file per resource (listings.js, orders.js ...)
    features/       auth/, listings/, cart/, orders/, seller/, admin/   (slice + hooks per feature)
    components/     ui/, layout/, listings/, commerce/
    pages/          Home.jsx, Search.jsx, ListingDetail.jsx, ...
    hooks/          useDebounce.js, useAuth.js ...
    mocks/          mock JSON used in week 1
    styles/         tailwind.css, tokens
    utils/          formatPrice.js, validators.js
```

9.5 Design guidelines

- **Identity:** pick a distinctive palette and one display plus one body typeface; avoid the default template look. The site should feel curated and trustworthy, like a boutique rather than a classifieds list.
- **Images first:** large square photo cards, fast gallery, consistent aspect ratios.
- **Signature flow:** a clean "upload photos, review, publish" flow on /sell/new that visibly leaves space for AI suggestions.
- **Accessibility and speed:** semantic HTML, focus states, alt text, lazy-loaded images, code splitting by route.

9.6 Phase 1 exit criteria

- All Must pages are built, responsive, and navigable with mock data.
- Component library is reusable and documented in a short README.
- Forms validate; loading, empty, and error states exist; routes are protected by role (mock auth).
- Project runs with one command and is committed to Git with a clear README.

10. Backend specification (Phase 2: Node.js and Express)

10.1 Technology

Concern	Choice
Runtime / framework	Node.js LTS, Express
Database access	Mongoose (ODM) with MongoDB Atlas
Authentication	JWT access + refresh tokens, bcrypt password hashing
Validation	Zod or express-validator
File uploads	Multer + Cloudinary (or S3-compatible) upload
Security middleware	helmet, cors (allowed origins only), express-rate-limit, mongo-sanitize
Logging / errors	morgan or pino, central error handler
Testing	Jest + Supertest, MongoDB Memory Server
API docs	Postman collection or Swagger/OpenAPI

10.2 Architecture and folder structure

Layered design: **routes** map URLs to **controllers**; controllers call **services** that hold business rules; services use **models**. Controllers stay thin and services are unit-testable.

```
backend/
  src/
    config/         env.js, db.js, cloudinary.js
    models/         User.js, Listing.js, Order.js, Review.js ...
    routes/         auth.routes.js, listings.routes.js, orders.routes.js ...
    controllers/    thin request/response handlers
    services/       listingService.js, orderService.js, aiClient.js ...
    middleware/     auth.js, requireRole.js, validate.js, upload.js, errorHandler.js, rateLimit.js
    validators/     zod/express-validator schemas
    utils/          ApiError.js, asyncHandler.js, pagination.js
    jobs/           cleanup of stale reservations (optional)
    app.js, server.js
  tests/            api and service tests
  scripts/          seed.js
```

10.3 Key backend behaviors

- **Auth flow:** login returns a short-lived access token and a refresh token (stored hashed); /auth/refresh rotates tokens; logout revokes.
- **Order placement:** inside a MongoDB transaction, check every cart listing is active, set it to reserved, create the order (one per seller), clear the cart, and create notifications. If any step fails, nothing is changed.
- **Stale reservations:** a pending order that is cancelled returns the listing to active; an optional job releases unpaid reservations after a timeout.
- **Ratings:** adding a review updates the seller ratingAvg and ratingCount.
- **AI gateway (Phase 4):** aiClient calls the AI service with a timeout of a few seconds; on failure it returns an empty draft so the seller continues manually.
- **Authorization:** middleware checks the role; services also check ownership (a seller edits only their own listing).

10.4 Environment variables (names only)

```
PORT, NODE_ENV, MONGODB_URI, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET,
ACCESS_TOKEN_TTL, REFRESH_TOKEN_TTL, CLIENT_ORIGIN,
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET,
SMTP_HOST, SMTP_USER, SMTP_PASS, AI_SERVICE_URL, AI_SERVICE_TOKEN, PLATFORM_FEE_PERCENT
```

10.5 Phase 2 exit criteria

- All endpoints in Section 8.2 (except AI) work and are documented in Postman or Swagger.
- Role and ownership rules are enforced and tested.
- Order creation is atomic and cannot sell the same item twice (verified by a concurrency test).
- Automated tests cover auth, listings, cart, and orders.

10.6 Phase 3: database finalization and integration (Week 4)

- Finalize schemas and indexes from Section 7; run explain plans on the main queries.
- Write the seed script and load realistic demo data.
- Replace mock data in the frontend with real API calls; fix integration bugs.
- Run end-to-end checks of the buyer, seller, and admin flows; record the month-1 demo (Milestone M1).

11. Deployment and DevOps

11.1 Environments

Environment	Purpose	Notes
Local	Daily development	Frontend dev server, backend with nodemon, local or Atlas dev database, AI service on localhost
Staging (demo)	Supervisor demos and testing	Deployed copy with seed data; free-tier services are acceptable
Production (later)	Real users	Separate database, secrets, and domain; backups enabled

11.2 Suggested hosting (choose during Phase 5)

Component	Options
React frontend	Static hosting such as Vercel, Netlify, or Cloudflare Pages
Node/Express API	A container or Node host such as Render, Railway, or Fly.io
MongoDB	MongoDB Atlas (managed)
Images	Cloudinary or an S3-compatible bucket with a CDN
AI service	Docker container on a small VM or container host; model files stored with the image or loaded from object storage; Hugging Face Spaces is an option for demos

11.3 CI/CD and operations

- **Git workflow:** main branch always deployable; short-lived feature branches; pull requests with a checklist.
- **CI (for example GitHub Actions):** install, lint, run tests, build the frontend on every pull request.
- **Deployment:** automatic deploy of main to staging; manual promotion to production.
- **Configuration:** every secret in environment variables, never in Git; a documented .env.example file in each project.
- **Monitoring:** health endpoints for backend and AI service, error logging, and a simple uptime check.

12. AI model specification (Phase 4)

The AI layer is built after the month-1 web application exists. It is a separate Python service so models can be trained, replaced, and scaled without touching the web code. The guiding rule: **AI assists, the human decides**. Every suggestion is editable, labeled "AI suggested", and has a non-AI fallback.

12.1 Model catalog

ID	Feature	Input to output	Baseline to improved approach	Metrics (initial targets)
M1	Category and color (AI-03)	Photos to category top-3, color top-3	Baseline: CLIP zero-shot. Improved: linear classifier on CLIP embeddings, or fine-tuned CNN/ViT	Top-3 accuracy at least 90% category, 85% color; macro F1
M2	Brand (AI-03)	Photos (+ text) to brand top-5	Baseline: CLIP zero-shot against a brand list. Improved: add OCR of labels/logos and a classifier for the top 30 to 50 brands	Top-3 accuracy at least 60% on supported brands; always allow manual entry
M3	Condition grading (AI-05)	Photos to new_with_tags, like_new, good, fair	Baseline: majority class. Improved: fine-tune a pretrained vision model on labeled marketplace images	Macro F1 at least 0.70; adjacent-class errors reported separately
M4	Title and description (AI-04)	Predicted attributes + seller notes to text	Baseline: fill-in templates. Improved: image captioning model plus a language model prompted with attributes	Human rating at least 4/5 on 50 samples; seller edit rate
M5	Price suggestion (AI-06)	Category, brand, condition, size, text, image embedding to price and range	Baseline: median price by category, brand, condition. Improved: gradient boosting (XGBoost or LightGBM) with text and embedding features	MAE and RMSLE better than baseline; at least 60% of predictions within +/-25% of actual sold price
M6	Semantic search and similar items (AI-01, AI-02)	Text query or image to ranked listings	Baseline: keyword text index. Improved: CLIP text and image embeddings with a vector index (FAISS or Atlas Vector Search)	Recall@10 and NDCG@10 on 50 to 100 labeled queries; must beat keyword search
M7	Fake / duplicate detection (AI-07)	Photos + price to duplicate score and flags	Perceptual hash plus embedding similarity plus price-outlier rules; optional classifier later	Precision at least 0.90 at recall at least 0.70 on synthetic and real duplicates

The targets above are starting goals to be confirmed after the baseline results exist. Report what the models actually achieve; honest numbers with a baseline comparison are more valuable to employers than inflated claims.

12.2 Datasets and data sources

Pretrained models reduce the amount of data needed, but evaluation and fine-tuning still need labeled data. Candidate sources are listed for prototyping. **Check every dataset and pretrained-model license before commercial use**; many academic datasets allow non-commercial research only.

Source	Useful for	Notes
Fashion Product Images dataset (Kaggle)	Category, color, usage labels (M1), embeddings (M6)	Tens of thousands of clean product photos; studio images differ from phone photos, so mix with real photos
DeepFashion (academic)	Category and attribute research, retrieval (M1, M6)	Typically limited to non-commercial research use; verify
iMaterialist Fashion (Kaggle / FGVC)	Fine-grained fashion attributes (M1)	Competition data terms apply; verify
Mercari Price Suggestion (Kaggle)	Text-based price modeling pipeline (M5)	Text and metadata only, no photos; competition terms apply; prices are from a different market, so re-calibrate on local sold data
Own marketplace data (seed + pilot)	Condition (M3), price (M5), feedback for all models	**Best for commercial use.** Collect real listings, photos, and accepted or edited AI suggestions with user consent

Source	Useful for	Notes
Self-labeled set	Condition grading and the golden evaluation set	Label with a written guideline; target 250 to 500 images per condition class and a 100-item golden test set

Do not scrape other marketplaces for training data without permission: their terms of service and image copyrights usually forbid it. Use public datasets, your own data, and volunteers or partner sellers instead.

12.3 Data preparation

- **Collect and clean:** remove corrupt files, tiny images, and exact duplicates; resize to a fixed input size; strip EXIF metadata (including location).
- **Label:** use a spreadsheet or a labeling tool such as Label Studio; write a one-page labeling guideline per task and measure agreement on a small shared sample.
- **Split:** 70% train, 15% validation, 15% test. Split by seller or item so the same item never appears in two sets (prevents leakage). Keep the test set untouched until the final evaluation.
- **Balance and augment:** handle class imbalance with weighting or resampling; light augmentation (crop, flip, color jitter) for training images only.
- **Version:** keep dated dataset folders or use DVC; record the dataset version in every experiment.

12.4 Training and evaluation plan

- **Environment:** Python 3.10+, PyTorch, Hugging Face Transformers, scikit-learn, XGBoost or LightGBM, FAISS; GPU on Colab, Kaggle, or company hardware.
- **Baseline first:** implement the simple baseline for every model and record its metrics before building anything complex.
- **Experiments:** fix random seeds, log hyperparameters, metrics, and dataset version (Weights and Biases, MLflow, or a simple CSV log); change one thing at a time.
- **Evaluation:** report metrics on the validation set during development and on the untouched test set once; include confusion matrices, example failures, and per-class results.
- **Model card:** for each model, a short document with purpose, data, metrics, limits, and known failure cases.
- **Model selection:** choose the smallest model that meets the target; speed and cost matter for a student-run product.

12.5 Serving and integration

- FastAPI service loads models at startup and exposes the endpoints in Section 8.3; each response includes model versions.
- The Node API calls the service with a timeout and an internal token; on timeout or error it returns an empty draft (feature degrades gracefully).
- Embeddings are computed when a listing is published and stored in MongoDB (or a vector index) so search does not recompute them.
- Package the service with Docker; optionally export heavy models to ONNX or quantize them for faster CPU inference.
- Latency budget: under 5 seconds for a full listing analysis; cache repeated requests.

Example /analyze response (shape only)

```
{
  "category":  [{"label": "jackets", "score": 0.91}, {"label": "coats", "score": 0.05}],
  "color":     [{"label": "black", "score": 0.88}],
  "brand":     [{"label": "<brand>", "score": 0.62}],
  "condition": {"label": "good", "score": 0.74},
  "title":       "<generated title>",
  "description": "<generated description>",
  "priceSuggestion": {"value": 0, "low": 0, "high": 0, "currency": "<store currency>"},
  "modelVersions": {"classifier": "m1-v1", "condition": "m3-v1", "price": "m5-v1"}
}
```

12.6 Monitoring and the feedback loop

- Log (with consent and no personal data) whether sellers accepted, edited, or rejected each AI suggestion.
- Track the acceptance rate and median edit size per feature; falling quality shows up here first.
- Retrain when enough new labeled data exists or when metrics drift; deploy new versions behind a version flag and compare against the current model.

12.7 Responsible AI

- **Transparency:** label suggestions as AI-generated; show confidence for classifications.
- **Control:** users can edit or ignore any suggestion; moderation decisions on flagged items are made by a human admin.
- **Privacy:** do not train on user photos without consent; remove location metadata; avoid images with faces where possible.
- **Fairness and safety:** test on diverse items and photo qualities; do not use the price model to discriminate between sellers; filter unsafe or prohibited items.

13. Development phases and plan

The schedule follows the company method: frontend first, backend next, then database, with the AI model after the first month. Durations for the AI phases are proposals and can be scaled to the internship length. Each phase has deliverables and exit criteria; a phase is finished only when its exit criteria are met.

Phase	Name	Timing	Main output
0	Setup and planning	Week 1, days 1-2	Repos, tooling, approved SRS, wireframes, design tokens
1	Frontend (React)	Week 1	All buyer and seller pages with mock data
2	Backend (Node / Express)	Weeks 2-3	Complete REST API with auth, roles, orders, tests
3	Database and integration	Week 4	Final schema, indexes, seed data, frontend on real API; **Milestone M1: month-1 demo**
4A	AI data collection and preparation	Weeks 5-6	Cleaned, labeled, versioned datasets; baselines
4B	AI model training and evaluation	Weeks 7-9	Trained models M1-M7 with metrics and model cards
4C	AI service and integration	Weeks 10-11	FastAPI service wired into listing, search, and admin flows
5	Testing, deployment, documentation	Week 12	Deployed demo, test report, README, demo video; **Milestone M2**

Phase 0: Setup and planning (Week 1, days 1-2)

- Create Git repositories (or one monorepo with frontend, backend, and ai-service folders) and a branching rule.
- Install and verify tools: Node.js LTS, Git, VS Code, MongoDB Atlas account, Cloudinary account, Postman.
- Approve this SRS with your supervisor; decide the open items in Section 17.
- Sketch wireframes for the main pages; choose color palette, typography, and spacing scale (design tokens).

Phase 1: Frontend with React (Week 1)

Objective: Build a complete, polished, responsive user interface using mock data so that backend work can start from a finished UI.

Task / step	Details
Day 1	Vite + React project, Tailwind config with design tokens, routing, layout components (Navbar, Footer), basic UI components (Button, Input, Modal, Toast).
Day 2	Home page, listing card and grid, search results page with FilterSidebar and sorting, category page, pagination, loading and empty states.
Day 3	Listing detail page with gallery, seller card, related items; cart page and cart state; checkout form and order confirmation page.
Day 4	Login and register pages with form validation, mock auth with protected and role routes, wishlist, profile settings, Create listing page with ImageUploader and AIDraftPanel placeholder.
Day 5	Seller dashboard and seller orders UI, responsive polish (mobile first), accessibility pass, README, demo with mock data, commit and review.

Exit criteria: Section 9.6 satisfied; every Must page works on mobile and desktop with mock data.

Phase 2: Backend with Node.js and Express (Weeks 2-3)

Objective: Build a secure REST API that implements all functional requirements except AI, with tests.

Task / step	Details
Week 2, days 1-2	Express project structure, config, error handling, logging; Mongoose connection; User model; register, login, refresh, logout; auth and role middleware.

Task / step	Details
Week 2, days 3-5	Categories, Listings CRUD, image upload to Cloudinary, search with filters, sorting, pagination; seller listing endpoints; public seller profile.
Week 3, days 1-2	Cart and wishlist endpoints; order placement with transaction and reservation; order status updates; order cancellation.
Week 3, days 3-4	Reviews and ratings; notifications; reports; admin endpoints (users, listings, categories, reports, stats); rate limiting and security middleware.
Week 3, day 5	Automated tests (auth, listings, cart, orders), Postman collection or OpenAPI docs, stub endpoint /ai/listing-draft that returns an empty draft.

Exit criteria: Section 10.5 satisfied; all non-AI endpoints pass tests; the API is documented.

Phase 3: Database and integration (Week 4)

Objective: Finalize the database and connect the frontend to the real API so the whole application works end to end.

Task / step	Details
Days 1-2	Finalize Mongoose schemas and indexes (Section 7); inspect query plans; write the seed script with 150 realistic listings and images.
Days 3-4	Replace mock data with real API calls; token refresh handling; error states; fix integration bugs; connect the admin pages.
Day 5	End-to-end testing of buyer, seller, and admin flows; fix bugs; deploy a staging build; record the demo. **Milestone M1.**

Exit criteria: A new user can register, a seller can list an item, a buyer can purchase it, the seller can fulfill it, and an admin can moderate it, all on real data.

Phase 4A: AI data collection and preparation (Weeks 5-6)

Objective: Gather and prepare the data needed to train and evaluate each model, and establish baselines.

Task / step	Details
Collect	Download candidate public datasets (Section 12.2) and check licenses; export seed and pilot listings from the app; collect real photos with consent.
Clean and label	Remove bad and duplicate images; strip metadata; create labeling guidelines; label condition grades and the 100-item golden test set.
Split and version	Train/validation/test split by item or seller; version the datasets; document data sources and licenses.
Baselines	Implement and score the baseline for each model M1-M7 (zero-shot CLIP, median price, keyword search, hash duplicates).

Exit criteria: Datasets are versioned and documented; baseline metrics exist for every model.

Phase 4B: AI model training and evaluation (Weeks 7-9)

Objective: Train, tune, and evaluate the models so that each beats its baseline or has an honest explanation if it does not.

Task / step	Details
Week 7	M1 category and color; M6 embeddings and vector index; evaluate search and similarity.
Week 8	M3 condition grading; M5 price model; M2 brand model (stretch, with fallback to manual input).
Week 9	M4 title and description generator; M7 duplicate detection; final test-set evaluation; error analysis; model cards.

Exit criteria: Each model has test-set metrics, a baseline comparison, an error analysis, and a model card.

Phase 4C: AI service and integration (Weeks 10-11)

Objective: Serve the models and connect them to the marketplace.

Task / step	Details
Service	FastAPI service with /analyze, /price, /embed, /search, /check-duplicate, /health; Dockerfile; load tests for latency.
Integration	aiClient in the Node API with timeout and fallback; AIDraftPanel in the Create listing page; semantic search box and Similar items; admin flagged queue.
Feedback	Log accepted and edited suggestions; add the AI suggested label; embeddings computed on publish.

Exit criteria: A seller can upload photos and receive an editable draft in under 5 seconds; semantic search and similar items work; the app still works if the AI service is down.

Phase 5: Testing, deployment, and documentation (Week 12)

Objective: Make the project demonstrable, reliable, and presentable.

Task / step	Details
Testing	Run the acceptance tests (Section 14), cross-browser and mobile checks, security checklist (Section 15).
Deploy	Deploy frontend, API, database, and AI service; configure environment variables; run seed script.
Document	README for each part, architecture diagram, model cards, results table, API docs, 2-minute demo video, short case study. **Milestone M2.**

Exit criteria: Live demo link works; all Must requirements pass; documentation is complete.

14. Testing and acceptance

14.1 Test levels

Level	What is tested	Tools
Unit	Services, utilities, validators, reducers	Jest (backend), Vitest or Jest (frontend)
API / integration	Endpoints with a test database, role checks, error cases	Jest + Supertest
Component / UI	Key components and forms	React Testing Library
End-to-end	Buyer, seller, and admin journeys in a real browser	Playwright or Cypress
AI evaluation	Metrics on the held-out test set; latency; fallback behavior	Python scripts, saved reports
Non-functional	Performance, accessibility, security basics	Lighthouse, manual checklist

14.2 Key acceptance tests

ID	Scenario	Expected result
AT-01	Register with an existing email	Rejected with a clear message; no duplicate user
AT-02	Buyer calls a seller-only endpoint	403 response; no data changed
AT-03	Seller creates a listing with 3 images	Listing is saved with status active and images visible in search
AT-04	Search with filters (size, price range, condition)	Only matching active items are shown; filters appear in the URL
AT-05	Two buyers check out the same item at the same time	Exactly one order succeeds; the other receives a conflict message
AT-06	Buyer cancels an order before shipping	Order cancelled; listing returns to active; notifications sent
AT-07	Order is completed, buyer rates seller	One review saved; seller average rating updated; second review blocked
AT-08	Admin removes a listing	Listing disappears from search; seller notified; order history unaffected
AT-09	Upload a 12 MB or non-image file	Rejected with a validation error
AT-10	AI service stopped, seller opens Create listing	Manual form works; no crash; friendly notice
AT-11	Upload a jacket photo (Phase 4)	Suggestions returned in under 5 seconds with category, color, condition, price, title; all editable
AT-12	Upload a photo that duplicates an existing listing (Phase 4)	Listing flagged for admin review

14.3 Definition of done

- Requirements for the phase are implemented and demonstrated.
- Code reviewed, formatted, and merged to main; no known critical bugs.
- Tests for the phase pass; documentation updated; demo recorded or shown to the supervisor.

15. Security checklist

Area	Control
Authentication	bcrypt hashing; strong password rules; token expiry; refresh token rotation and revocation; generic login errors
Authorization	Role middleware on every protected route; ownership checks in services; no trust in client-provided roles or ids
Input handling	Server-side validation for every body, query, and param; sanitize against NoSQL injection; escape output to prevent XSS
Transport	HTTPS only; secure, httpOnly cookies if cookies are used; CORS restricted to the frontend origin
Headers and limits	helmet headers; rate limiting on auth and write routes; request body size limits
Uploads	Type and size checks; random file names; images served from the storage provider, not the API server
Secrets	Environment variables only; .env never committed; rotate keys if leaked
Dependencies	Lockfiles; regular audit of npm and pip packages; remove unused packages
AI service	Not publicly reachable or protected by an internal token; request size limits; no raw user data in logs
Data protection	Minimal personal data; account deletion path; backups protected; EXIF location removed from images

16. Risks and mitigation

Risk	Impact	Mitigation
Not enough training data for condition or brand	Weak model accuracy	Start with pretrained CLIP and linear classifiers; focus on fewer brands; keep manual fallback; collect data from the pilot
Dataset license prevents commercial use	Cannot monetize	Track licenses in a table; prefer own data and permissive models for production; keep academic datasets for evaluation only
Scope creep in the first month	MVP not finished	Freeze scope at the Must items; park new ideas in Section 18
Underestimating frontend effort	Week 1 overrun	Build Must pages first; reuse components; admin pages later
Same item sold twice	Broken trust	Atomic reservation and a concurrency test (AT-05)
AI service slow or down	Poor experience	Timeouts, fallback to manual form, caching, smaller models
Free-tier limits (storage, hosting, GPU)	Blocked work	Compress images, use Colab/Kaggle for training, keep data small, plan paid tier before launch
Fake or prohibited listings	Safety and reputation	Reports, duplicate detection, admin moderation, clear listing rules
Photo quality varies (phone photos vs studio)	Lower AI accuracy	Include real phone photos in training and evaluation; apply augmentation
Time lost on tool setup	Delays	Phase 0 setup; use well-known defaults from this SRS

17. Development handoff guide

This section is for the moment you hand this SRS to your AI assistant (or a teammate) and start building. Confirm the decisions below first, then start one phase at a time.

17.1 Decisions to confirm before Phase 1

Decision	Default in this SRS	Your choice
Product name and logo	Working title only	
Visual style and palette	To be chosen in Phase 0	
Currency, language, locale	Configurable currency; English UI	
Frontend state / data fetching	Redux Toolkit (+ RTK Query or TanStack Query)	
Image storage	Cloudinary	
Hosting targets	Vercel/Netlify, Render/Railway, Atlas	
Payment in v1	Cash on delivery + test card (simulated)	
Platform fee percentage	Configurable (PLATFORM_FEE_PERCENT)	
Starting categories	3 top-level fashion categories with subcategories	
GPU access for Phase 4	Colab or Kaggle	

17.2 Prompt template to start a phase

```
I am starting Phase <number and name> of the AI Resale Marketplace project.
Use the attached SRS as the source of truth.
Relevant sections: <e.g. 4, 9, 13>.
Decisions made: <paste the filled table from 17.1>.
Already completed: <previous phases / what exists in the repo>.
Task now: <e.g. "Build Day 1 and Day 2 of Phase 1">.
Deliver: working code with file structure, run instructions, and a short
checklist mapped to the exit criteria of this phase.
```

17.3 Rules for working phase by phase

- Work in the order of Section 13; do not start the next phase until the exit criteria are met.
- Keep the API contract in Section 8 stable; if it must change, update the SRS and bump the version.
- Always keep the manual (non-AI) path working; AI only adds suggestions.
- Commit small and often; each requirement ID should map to a commit or pull request when possible.

18. Future enhancements

- Real payment gateway, seller payouts, and escrow-style buyer protection.
- Courier integrations, shipping labels, and live tracking.
- Real-time buyer-seller chat and offers (make an offer / counter offer).
- Native mobile app or installable PWA with push notifications.
- More categories (accessories, shoes, home goods) with category-specific models.
- Visual search: upload a photo to find similar items.
- Size and fit recommendations; outfit suggestions from a user's wishlist.
- Seller analytics and promoted-listing tools; subscription plans.
- Email and WhatsApp notifications; multi-language support.
- Authenticity checks for premium items and a trust score for sellers.

Appendix A: Traceability matrix (modules to phases)

Module	P1 Frontend	P2 Backend	P3 Database / integration	P4 AI
Authentication and users	Login, register, profile UI	Auth endpoints, JWT, roles	User schema, indexes	-
Listing management	Create/edit form, uploader	Listings CRUD, upload	Listing schema, seed data	AI-03 to AI-06 draft
Browse and search	Feed, search, filters UI	Search, filter, sort API	Text index, compound indexes	AI-01, AI-02
Cart and wishlist	Cart and wishlist UI	Cart/wishlist endpoints	Carts, wishlists collections	-
Checkout and orders	Checkout, tracking UI	Atomic order placement	Order schema, transactions	-
Seller dashboard	Dashboard UI	Seller endpoints, stats	Aggregation queries	Price hints
Reviews and ratings	Rating UI	Review endpoints, averages	Review schema, unique index	-
Notifications	Notification center UI	Notification endpoints	Notification schema	-
Admin panel	Admin pages (after backend)	Admin endpoints	Reports schema	AI-07 flag queue
AI module	AIDraftPanel placeholder	Stub /ai endpoints	embedding, aiMeta fields	All seven features